TECHNICAL SESSION · GITHUB COPILOT

Spending Premium Requests Wisely

Token & premium-request economics, context features, custom instructions, prompt files, MCP, and the workflow patterns that make Copilot fast, cheap, and good.

Use ← → or Space to navigate · Press F for fullscreen

SET THE STAGE

Why Copilot users should care about tokens



PREMIUM REQUESTS

Business / Enterprise plans cap monthly premium requests. Heavy users hit limits by the third week.



LATENCY

Bigger context = slower responses. Agent mode on a huge repo can take 15–30s.



QUALITY

"Context rot" — model accuracy drops as the window fills, even within limits.



AGENT MODE

Agent loops **multiply** requests. Sloppy prompts can burn 30 requests on one task.

Copilot abstracts dollars away — but premium-request budget is the same lever. Spend it on hard problems, not on context the model didn't need.

KNOW YOUR SURFACES

Where Copilot reads context

SURFACE	WHAT IT SENDS	TOKEN WEIGHT
Inline completions (ghost text)	Open file + nearby files (neighboring tabs)	Light · auto-managed
Copilot Chat	Active file, selection, attached `#file` / `#selection`, history	Medium · you control it
Agent mode (broad retrieval)	Indexed repo chunks via retrieval + tool calls	Heavy · scales with repo size
Edits / Agent mode	Working set + tool calls + multi-turn loop	Heaviest · multiplies per iteration
Coding agent (cloud)	Issue + repo + CI logs across many turns	Heavy · long-running
Copilot CLI	Shell context + files you reference	Light–medium

Different surface, different optimization. Treat them differently.

LIVE CALCULATOR

Premium-request budget calculator

Model (multiplier per request)

GPT-5 — 1×

Chat requests / day / dev

40

Agent runs / day / dev

3

Avg tool turns per agent run

8

Developers

50

Plan (monthly premium-request quota / user)

Business — 1500

Premium requests / dev / day —

Premium requests / dev / month (22d) —

% of monthly quota used —

Team total / month —

Adjust the model or workload to see how the budget moves.

Multipliers per GitHub's published premium-request table. Agent runs count each tool turn that calls the model. Adjust to your reality.

ROADMAP

Copilot-native levers we'll cover

A

Scoped context

#file, #selection, #codebase, attachments

B

Custom instructions

copilot-instructions.md + scoped *.instructions.md

C

Prompt files

Reusable .prompt.md templates

D

Spaces & AGENTS.md

Curated context bundles

E

Model picker

Right-size per task; protect premium budget

F

MCP & tool hygiene

Trim tools; targeted context vs broad retrieval

G

Repo hygiene

.gitignore for the indexer; symbol-friendly code

H

Workflow patterns

Ralph loop, auto-compact, agent handover

LEVER A

Scoped context — the #1 Copilot win

Most teams overuse broad retrieval (Agent mode for simple questions). It's powerful but expensive. Reach for sharper variables first.

✅ Use targeted variables

- `#file:src/api/orders.ts` — exactly the file
- `#selection` — only what's highlighted
- `#editor` — visible portion only
- `#terminalLastCommand` / `#terminalSelection`
- `#problems` — the diagnostics, not the whole file
- `#fetch:<url>` — pull docs on demand
- `#changes` — git diff vs base

❌ Don't reflexively reach for

- Agent mode + a one-line question — retrieves & dumps a lot of context
- `#codebase` when one `#file` would do
- Pasting whole stack traces — use `#problems` or trim
- "Look at all the test files" — attach the one you mean

LEVER B

Custom instructions — codify it once

Anything you tell Copilot in *every* chat belongs in an instructions file. It loads automatically — no tokens spent re-typing.

Repo-wide

```
.github/copilot-instructions.md
```

Build/test/lint commands, naming conventions, "be concise, no preamble", which dirs to ignore, output style. Auto-applied to every chat in the repo.

Scoped (path-targeted)

```
.github/instructions/react.instructions.md
---
applyTo: "src/**/*.tsx"
---
Use functional components.
Prefer hooks over HOCs.
Tests live next to the component.
```

Loaded only when files matching the glob are in context. Smaller per-turn footprint.

Personal-level instructions live in VS Code settings — use these for your style preferences, not team standards.

LEVER C

Prompt files — reusable, parameterized

For repeating tasks (write a test, scaffold a route, run a migration), a saved `.prompt.md` beats re-typing the same prompt every time.

Where they live

```
.github/prompts/new-endpoint.prompt.md
```

Run with `/new-endpoint` in chat. Versioned with the repo so the whole team uses the same recipe.

Why it saves tokens

- One canonical prompt → consistent prefix → better cache hit rate.
- Encodes "do this *exactly* this way" — model doesn't wander, fewer retries.
- Embeds `#file` references, so context is curated once.
- Ship them in your repo template; new devs are productive on day one.

LEVER D

Copilot Spaces & AGENTS.md

Copilot Spaces

Curated bundle of files, docs, and instructions for a topic (e.g. "Payments domain", "Onboarding"). The Space is loaded as the chat's working context.

- Replaces "let me drag in 12 files every time."
- Shareable across the team.
- Smaller, focused than broad Agent-mode retrieval.

AGENTS.md

The cross-tool standard (Copilot, Claude Code, Cursor, Aider all read it). One file in repo root telling agents:

- How to build, test, lint
- Conventions and forbidden patterns
- Where docs live
- "Don't touch these dirs"

Combine with `copilot-instructions.md` — Copilot reads both.

LEVER E

The model picker is your routing knob

MODEL	MULTIPLIER	REACH FOR IT WHEN…
GPT-4.1	0× (included)	Boilerplate, completions, simple Q&A. Default if it fits.
GPT-5 mini / o3-mini	0.33×	Lightweight chat, quick reasoning, cheap iterations.
GPT-5	1×	Default for "real" coding work, debugging, design.
Claude Sonnet 4 / 4.5	1×	Long contexts, agent mode, careful refactors.
Gemini 2.5 Pro	1×	Very long files, multimodal needs.
Claude Opus 4	**10×**	Genuinely hard reasoning. Use sparingly — one Opus call = ten Sonnet calls.

Multipliers per GitHub's published list (May 2026). Verify in your billing settings.

Rule: start with the cheapest viable model. Escalate only when stuck. Most "I need Opus" feelings disappear with better context.

LEVER F

MCP servers & chat participants

MCP hygiene

- VS Code Copilot supports MCP — every connected server's tools are sent on every turn.
- Disable servers you don't use this session (settings → MCP).
- Prefer servers that return **summarized** data, not raw dumps.
- Audit `.vscode/mcp.json` in the repo — keep it minimal.

Participants & retrieval — pick the right scope

- **Agent mode** — repo-wide retrieval + tools. **Heavy.**
- `@vscode` — IDE help only. Cheap.
- `@terminal` — shell context. Cheap.
- `@github` — issues/PRs. Targeted.
- Ask mode + targeted `#file` — usually the cheapest path.

LEVER G

Repo hygiene for Copilot's indexer

Make retrieval cheap

- `.gitignore` drives what Copilot indexes — keep it tight (no `target/`, generated code, large fixtures).
- Use **content exclusions** in repo / org settings to keep secrets and noise out of context.
- Smaller, well-named modules → retrieval finds the right symbol.
- Keep README and per-folder docstrings current — a 200-token summary beats re-reading 5,000 tokens of source.

Make completions sharper

- Open the files Copilot should consider — **neighboring tabs** are inline-completion fuel.
- Close noisy unrelated tabs.
- Name things well — Copilot leans heavily on identifiers.
- Type the function signature first; let inline completions fill the body.

LEVER H · WORKFLOW PATTERNS

Three patterns for directed Copilot workloads

Especially in **Agent mode** and the **Copilot coding agent**, structure beats raw model power.

1

The Ralph Wiggum loop

One simple loop, run repeatedly, against a checklist file.

2

Auto-compact

Bound the chat. Start fresh per task.

3

Agent handover

Specialized prompt files / chats passing baton via files.

PATTERN 1

The Ralph Wiggum loop — Copilot edition

"Me fail English? That's unpossible." Trade brain for repetition: **one cheap, simple loop** instead of one elaborate clever prompt.



How to run it in Copilot

- Save it as `.github/prompts/ralph.prompt.md` + invoke with `/ralph`.
- Use **GPT-4.1 or Sonnet** — cheap models, many cheap iterations.
- State lives in `TODO.md`, not chat history → small fresh context every turn.
- Excellent fit for Copilot **Agent mode** and the **cloud coding agent**.

The loop prompt

```
1. Read TODO.md
2. Pick the first unchecked item
3. Implement it (use #codebase only if needed)
4. Run tests; if green, check the box
5. If red, write what failed and stop
6. Commit
GOTO 1
```

PATTERN 2

Auto-compact in Copilot Chat

Copilot doesn't auto-summarize as aggressively as Claude Code. **You have to do it.**

How to compact manually

- At a natural break, ask: "*Summarize what we decided, what files changed, and the next step. Keep it under 300 words.*"
- Open a **new chat**; paste the summary as the first message.
- Or: save the summary into `plan.md`; reference it in new chats with `#file:plan.md`.
- `/clear` resets chat history — pair with the summary file.

Why it matters more in Agent mode

- Agent loops accumulate tool output fast — a 5-step task can hit 50k tokens.
- Each subsequent turn re-pays that cost (modulo cache).
- Quality drops well before the hard window limit (context rot).
- **Rule of thumb:** one task per chat. New task → new chat.

PATTERN 3

Agent handover with Copilot

Instead of one mega-chat, run a **relay** of specialized prompt files that hand off via files in the repo.

Planner
/plan · GPT-5 · writes plan.md

→

Implementer
/build · Sonnet/4.1 · works through plan.md

→

Reviewer
/review · GPT-5 · checks diff vs plan

Why it saves premium requests

- Each agent has only the context it needs — planner doesn't see code, implementer doesn't re-derive the plan.
- Cheap models do cheap work; one expensive call up front for planning.
- The handover artifact (`plan.md` , diff, `review.md`) is the only thing crossing.
- Failures are localized — re-run one stage, not the whole conversation.

Handover hygiene

- Standardize the artifact format (sections, schema).
- Each hop = a **fresh chat** with the relevant prompt file.
- Include a "definition of done" the next agent can verify.
- Commit artifacts so a human can audit the trail.

PUTTING IT TOGETHER

A combined Copilot pipeline

/plan (GPT-5) → writes TODO.md → /ralph in Agent mode (Sonnet) → compact at 60% / one task per chat → /review (GPT-5) → PR via cloud coding agent

1× Planner

One GPT-5 call · produces TODO.md + acceptance criteria.

N× Implementer

Cheap iterations · each loop has small, fresh context.

1× Reviewer

Sees diff + criteria only · returns pass/fail + notes.

Net effect: dramatically fewer premium requests for the same outcome — and the trail is auditable.

CALL THESE OUT IN PR REVIEW

Copilot anti-patterns to stamp out

❌ Context

- Using Agent mode for every question.
- One chat that's been running for a week.
- Pasting whole files instead of attaching with `#file`.
- Connecting every MCP server "just in case."
- Leaving 30 unrelated tabs open while taking inline completions.

❌ Spend

- Defaulting to Claude Opus for everything (10× multiplier!).
- "Explain everything you're doing as you go" — output-token tax.
- Re-typing the same standards in every prompt instead of `copilot-instructions.md`.
- Running Agent mode without a clear plan → 30 tool turns of wandering.
- Ignoring premium-request usage in the GitHub billing dashboard.

SCALING — WHEN YOU HAVE 40 SKILLS, NOT 4

The 7-layer context system, mapped to Copilot

From a production repo running this pattern across ~40 skills. Each layer is independently portable into a Copilot workspace.

#	LAYER	WHERE IT LIVES IN YOUR COPILOT REPO
1	**Context budget planning**	Comments at top of `AGENTS.md` / each prompt file: model, fixed cost, per-turn budget
2	**Progressive disclosure**	`AGENTS.md` → `.github/skills/{name}/SKILL.md` → `references/`
3	**Context shredding**	Three skill variants: `SKILL.md` / `SKILL.digest.md` / `SKILL.minimal.md`
4	**Context-awareness tags**	`<context_awareness>` block inside each long prompt file or agent definition
5	**Subagent extraction**	Separate prompt files (`/cost-estimate` , `/test-runner`) invoked from a parent
6	**Glob targeting**	`applyTo:` in `.github/instructions/*.md` — never `**` unless universal
7	**Observability**	Latency as a proxy + npm lint scripts over `.github/`

LAYERS 1 – 3 — ARCHITECTURE INSIDE `.GITHUB/`

Budget the prompt, structure the skills, compress at runtime

1. Budget (design time)

```
# GPT-5 (128K context)
- Workspace info:    ~500
- copilot-instr.md:  ~400
- 5 *.instructions:  ~3,000
- Open editor file:  ~1,500
- #file mentions:    ~4,000
- Output headroom:  -16,000
_____________________________
Per-turn budget:   ~103,000
```

Hard limits at authoring time: prompt file ≤ 250 ln, ≤ 30 tools per chat, instructions ≤ 150 ln each.

2. Progressive disclosure

```
AGENTS.md                          # always loaded
.github/copilot-instructions.md    # always loaded
.github/instructions/*.md          # glob-matched
.github/skills/{name}/SKILL.md     # on-demand
.github/skills/{name}/references/  # deep content
```

`AGENTS.md` is a table of contents that points Copilot toward the right skill — never a comprehensive guide.

3. Context shredding — three sizes per skill

TIER	FILE	SIZE	WHEN COPILOT LOADS IT
Full	`SKILL.md`	400–800 tokens	Fresh chat, < 60% context
Digest	`SKILL.digest.md`	150–320 tokens	60–80%
Minimal	`SKILL.minimal.md`	40–100 tokens	> 80% — decisions only

A small Node script (`generate-skill-digests.mjs`) builds the digest + minimal variants from `SKILL.md`. Run it in CI; check the diffs.

Self-aware prompts, isolated chats, measurable bloat

4. Context-awareness tag in a prompt file

```
---
description: 'Build feature from plan'
mode: agent
---
<context_awareness>
Big prompt (~250 ln). At >60% context,
load SKILL.digest.md instead of SKILL.md.
At >80% stop re-reading prior turns;
ask user to checkpoint.
</context_awareness>
```

Long prompts get an explicit **compaction phase**: summarize what's been done, switch to minimal skills, then continue.

5. Subagent extraction = a fresh Copilot chat

```
Parent chat (planning):  5K tokens
  → opens NEW chat with /cost-estimate
    runs in fresh 128K window
    returns 2K JSON result
Parent continues:        +2K instead of +40K
```

Trigger when: > 5 tool calls for one subtask, domain shift, or chat > 60% full. Send the subagent a **focused brief**, not the whole conversation.

6. Glob targeting in `.github/instructions/`

```
# BAD — every Copilot turn pays for this
---
applyTo: "**"
---

# GOOD
applyTo: "src/**/*.ts"
applyTo: "**/*.bicep"
```

7. Observability — latency tells you the truth

COPILOT REPLY LATENCY	≈ INPUT TOKENS
< 5s	< 20K
5–10s	20–60K
10–20s	60–120K
> 20s	Near limit — start a new chat

TAKE-HOME

Quick-win checklist for Copilot teams

This week

- ☐ Commit `.github/copilot-instructions.md` — build, test, lint, "be concise".
- ☐ Add `AGENTS.md` at repo root.
- ☐ Tighten `.gitignore`; add content exclusions for secrets/generated code.
- ☐ Audit MCP servers — disable what you don't use.
- ☐ Switch default model to GPT-4.1 or GPT-5; reserve Opus for hard problems.

This month

- ☐ Build 3 prompt files for your most common tasks (`.github/prompts/`).
- ☐ Add scoped `*.instructions.md` with `applyTo` globs per stack.
- ☐ Adopt one-task-per-chat + manual compact via `plan.md`.
- ☐ Try the planner→implementer→reviewer relay on one workstream.
- ☐ Train the team on `#file`, `#selection`, `#problems` — kill Agent-mode-by-default.
- ☐ Review premium-request usage monthly.

FOR THE FACILITATOR

Suggested 60-minute session structure

MIN	SEGMENT	FORMAT
0–5	Why premium requests matter (budget, latency, context rot)	Talk + chart
5–15	Where Copilot reads context + live calculator	Demo
15–35	Levers A–G with one Copilot example each	Walkthrough
35–45	Workflow patterns: Ralph loop, compact, handover in Agent mode	Diagrams + demo
45–55	Hands-on: write copilot-instructions.md + 1 prompt file together	Pair exercise
55–60	Q&A + standards we will adopt	Discussion

FURTHER READING

References

GitHub Copilot docs

- Copilot — *Custom instructions for repositories*
- Copilot — *Prompt files* + `applyTo` frontmatter
- Copilot — *Premium requests* and model multipliers
- Copilot — *Spaces* overview
- Copilot — *Agent mode* and *Coding agent*
- VS Code — *MCP support in Copilot Chat*
- GitHub — *Content exclusions* for Copilot

Foundational

- Anthropic — *Effective context engineering for AI agents*
- Chroma Research — *Context Rot* benchmark
- Geoffrey Huntley — *The Ralph Wiggum agentic coding pattern*
- Cognition — *Don't build multi-agents* (sub-agent context isolation)
- `AGENTS.md` spec — agents.md

IN ONE SENTENCE

Treat premium requests as a budget, structure as the multiplier.

Targeted context, instructions instead of repetition, cheap loops, one task per chat — that's how a Copilot team stays fast, cheap, and good.

Questions?